

任务号	5
姓名	杨力泉 元杰
学号	231220069 231220082
日期	2026 年 6 月 2 日
选做任务	任务1：数据流分析相关优化

实验报告 — 基于数据流分析的全局优化

一、实验目标与概述

在实验三生成的 IR 基础上，使用数据流分析框架完成三种循环无关的全局优化：常量传播（含常量折叠）、全局公共子表达式消除（依赖可用表达式分析与复制传播）、全局无用代码消除（依赖活跃变量分析）。框架已实现 IR 解析、基本块划分、控制流图构建及统一求解器，需完成 5 个文件中的 TODO：后向求解器、活跃变量分析、常量传播、可用表达式分析与复制传播。

二、总体设计与数据流框架

优化流水线依次执行，后一步可利用前一步暴露的优化机会：

1. ConstantPropagation + constant_folding	前向 Must , 三态格
2. AvailableExpressionsAnalysis merge_common_expr → solve → remove_expr	前向 Must
3. CopyPropagation + replace_available_use	前向 Must
4. ConstantPropagation (第二轮)	前向 Must
5. LiveVariableAnalysis 迭代	后向 May , 直到无新死代码

每个分析实现统一的虚函数接口：

虚函数	语义
isForward	前向还是后向分析
newBoundaryFact	Entry/Exit 的特殊初始 Fact
newInitialFact	其余块的初始 Fact
meetInto(fact, target)	将 fact 合并入 target，返回是否变化
transferBlock(blk, in, out)	块内传递：遍历语句更新 Fact

分析	方向	May/Must	Fact 类型	meet
常量传播	前向	Must	Map[var → CPValue]	meetValue
可用表达式	前向	Must	Fact_set_var(含 is_top)	intersect
复制传播	前向	Must	Fact_def_use(def → use, use → def)	intersect

分析	方向	May/Must	Fact 类型	meet
活跃变量	后向	May	Set[var]	union

求解器在 `worklist_solver` 中根据 `isForward` 选择初始化与遍历方向。前向时 Entry 为 Boundary (`newBoundaryFact` 赋给 `OUT[entry]`)，后向时 Exit 为 Boundary (赋给 `IN[exit]`)。Worklist 算法仅在 `transferBlock` 返回变化时才将后继（前向）或前驱（后向）入队，避免不必要的重复计算。

三、具体实现

3.1 后向求解器（参照前向实现）

- **initializeBackward**：Exit 的 IN 赋 Boundary（空集），其余块 OUT/IN 均初始化为 `newInitialFact`（空集 = Bottom）
- **iterativeDoSolveBackward**：每轮遍历所有块，`OUT[blk] = meetAll(IN[succ])`，再 `transferBlock` 更新 `IN[blk]`；任一 IN 变化则继续
- **worklistDoSolveBackward**：同上逻辑，变化时将前驱加入 worklist 而非后继

3.2 活跃变量分析（后向 May 分析，死代码消除）

后向分析的核心原因：变量在某处被定义后，是否在后续被使用，自然应从后往前看。

- **meetInto**：`union` — 任一后继需要该变量，则当前出口该变量即为活跃
- **transferStmt**：`IN = use U (OUT - def)` — 先 kill（当前定义的变量不再需要前驱提供），再 gen（当前使用的变量需要前驱定义）
- 死代码消除：从 Exit 反向推进，若某 `IR_OP_STMT` 或 `IR_ASSIGN_STMT` 的 def 不在当前活跃集合中，说明此后不再被使用，标记 `stmt->dead = true`。因消除死代码可能暴露新的死代码，需迭代直到不再变化。

3.3 常量传播（前向 Must 分析，三态格）

三态格：**UNDEF < CONST < NAC**。UNDEF 表示未定义（Map 中不存在），CONST(k) 表示确定为常量 k，NAC 表示非常量。

- **meetValue**：`UNDEF ∪ V = V`（初始状态 UNDEF 遇到任何值取对方），`CONST1 ∩ CONST2 = CONST1 (if =) else NAC`，NAC 与任何值 meet 均为 NAC
- **calculateValue**：仅当两个操作数均为 CONST 时直接计算（`v1_const + v2_const` 等），除零返回 UNDEF；任一为 UNDEF 则结果 UNDEF；任一为 NAC 则结果 NAC
- **newBoundaryFact**：函数参数初始化为 NAC（外部传入值不确定）
- **transferStmt**：`IR_ASSIGN_STMT` → def 取 use 的 CPValue；`IR_OP_STMT` → def 取 calculateValue 结果；LOAD/CALL/READ → def 为 NAC（运行时才能确定）

3.4 可用表达式分析（前向 Must 分析，公共子表达式消除）

- **Fact 设计**：`Fact_set_var` 含 `is_top` 标记。TOP 表示全集（所有表达式均可用），用于初始化。Entry 的 OUT 为 $\emptyset$ (BOTTOM)，其余块初始化为 TOP
- **meetInto**：若 fact 为 TOP 则不变；若 target 为 TOP 则直接赋值为 fact（首次 meetInto 时发生）；否则 `intersect`——Must 分析要求所有前驱都有该表达式才可用
- **merge_common_expr**：框架已实现，对相同 (`op, rs1, rs2`) 的表达式分配统一的 `expr_var`，并将 `v := expr_var` 插入赋值链

- **remove_available_expr_def** : 若 `expr_var` 在 IN fact 中已存在, 说明表达式已在前驱计算过, 标记重复计算为 dead

3.5 复制传播 (前向 Must 分析)

- **Fact 设计** : `Fact_def_use` 含 `def_to_use` 和 `use_to_def` 双向映射, `is_top` 标记
- **meetInto** : Must 分析使用 intersect, 即目标中存在但 fact 中不存在的映射被删除
- **copy_kill** : 新定义的变量同时杀死 `def_to_use[new_def]` 和 `use_to_def[new_def]`
- **copy_gen** : `x := y` (`y` 非常量) 时插入 `def_to_use[x]=y` 和 `use_to_def[y]=x`

四、主要困难与解决方案

1. **后向分析方向理解** : 活跃变量从 Exit 反向计算, `pred` $\leftrightarrow$ `succ`、`IN` $\leftrightarrow$ `OUT` 反转即可。对照前向求解器的对称结构实现。
2. **常量 meet 逻辑** : 最易出错的是 `UNDEFuc=c` (非 `UNDEF`) , 否则 `UNDEF` 会"吞掉"常量导致分析失效。正确实现后, 所有路径初始 `UNDEF` 在首次遇到 `CONST` 时即为 `CONST`。
3. **可用表达式 TOP/BOTTOM** : Must 分析初始状态为 `TOP` (全集), `Entry` 的 `OUT` 为 `BOTTOM` ($\emptyset$) , 第一个前驱的 `meetInto` 通过 "target is TOP $\rightarrow$ 直接赋值" 绕过 `intersect`, 避免 $\emptyset$ 被过度保守地传播。

五、构建与运行

```
cd Code && make parser
./parser input.ir output.ir
diff input.ir output.ir
```